



WHITEPAPER

Die unsichtbare Architektur sichtbar machen

Abhaengigkeits-Landkarten als Mittel gegen Software-Verfall

Herausgeber: **FINLAI designs**

Linz, Oesterreich · financial-analytics.eu

Stand: 2026-06-05

Entwickelt in Linz, Oesterreich

Herausgeber: FINLAI designs · Linz, Oesterreich · financial-analytics.eu **Format:** Uebergreifende, erklärende, an wissenschaftliche Texte angelehnte (vorwissenschaftliche) Arbeit. Jeder Fachbegriff wird bei Erstnennung erläutert; jedes Kapitel beginnt mit einem Einstieg.

Zielgruppe: Management/IT-Leitung **und** technische Umsetzer — verstaendlich ohne tiefes Vorwissen. **Gegenstand:** Eine allgemeine, herstellernerneutrale Methodik, die verborgenen internen Abhaengigkeiten einer Codebasis sichtbar und steuerbar zu machen — die *Abhaengigkeits-Landkarte* (Dependency Map). **Hinweis zur Anonymisierung:** Methodik und Belege sind herstellernerneutral gehalten; der Fliesstext nennt bewusst keine Projekt-, Produkt- oder Personennamen. Konkrete Zahlen stehen anonymisiert in den Kaesten „Aus der Praxis“ und stammen aus *einer* realen Desktop-Anwendung mittlerer Groesse.

Kurzfassung

Software hat keine natuerliche Gestalt — man kann sie nicht ansehen wie ein Gebaeude. Ihre wichtigste Struktur, naemlich *wer haengt von wem ab*, bleibt im Quelltext verborgen und waechst mit jeder Aenderung unkontrolliert weiter. Diese Arbeit beschreibt eine konkrete Gegenmassnahme: die **Abhaengigkeits-Landkarte** (Dependency Map), die genau diese verborgene Struktur aus dem Code herausliest und sichtbar macht. Sie spannt den Bogen von den grossen, seit Jahrzehnten bekannten Grundproblemen der Softwareentwicklung (Vorwort) ueber den belegbaren Nutzen einer solchen Karte (Kapitel 5) bis zur praktischen Einrichtung des Werkzeugs (Kapitel 6) und einer ehrlichen Darstellung seiner Grenzen (Kapitel 7). Als durchgehendes, anonymisiertes Fallbeispiel dient eine reale Desktop-Anwendung mit rund 750 Modulen, fuer die eine solche Karte vier Import-Zyklen — darunter einen dicht verflochtenen Kern aus Datenbank- und Kryptographie-Modulen — sowie mehrere Schicht-Verletzungen aufgedeckt hat. Das Leitprinzip ist durchgaengig dasselbe: *Was man nicht sieht, kann man nicht steuern — und eine aus dem Code erzeugte Landkarte kann, anders als jedes handgemalte Bild, gar nicht erst veralten.*

1. Vorwort: die wiederkehrenden Grundprobleme der Softwareentwicklung

Einstieg: Bevor man ein Werkzeug versteht, muss man das Problem kennen, das es loest. Dieses Kapitel sammelt die grossen, seit Jahrzehnten bekannten Schwierigkeiten der Softwareentwicklung und zeigt, dass sie sich alle in einer einzigen, messbaren Groesse buendeln: den Abhaengigkeiten zwischen den Bausteinen eines Systems.

Bereits 1986 unterschied **Frederick Brooks** in seinem beruehmten Aufsatz *No Silver Bullet* zwei Arten von Schwierigkeit: die *essenzielle* (in der Natur der Software selbst liegende) und die *akzidentelle* (nur technologiebedingte). Zu den vier essenziellen Eigenschaften zaehlt er die **Unsichtbarkeit**: Software hat keine natuerliche geometrische Darstellung, kein Bauplan-Bild, das ihre Struktur unmittelbar

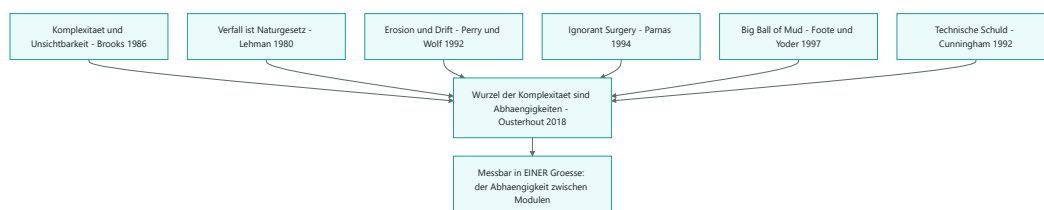
zeigt. Brooks' Befund ist nüchtern — diese Unsichtbarkeit lässt sich nicht vollständig aufheben. Eine Abhängigkeits-Landkarte beseitigt sie also nicht, aber sie *mildert* eine ihrer wichtigsten Dimensionen, indem sie eine bestimmte verborgene Struktur — den Import-Graphen — sichtbar und besprechbar macht.

Dass diese Struktur sich ohne Gegenwehr verschlechtert, ist kein Versagen einzelner Teams, sondern beinahe ein Naturgesetz. **Meir Lehman** formulierte hierzu seine *Gesetze der Software-Evolution*: ein genutztes System muss ständig angepasst werden (*Continuing Change*), und mit jeder Anpassung steigt seine Komplexität und seine Struktur verfaellt — *es sei denn, es wird aktiv Arbeit investiert, um sie zu erhalten (Increasing Complexity)*. Genau hier setzt regelmäßige Messung an: Wer den Verfall nicht sieht, investiert diese Arbeit nicht.

Dewayne Perry und Alexander Wolf gaben dem Verfall 1992 zwei präzise Namen. *Architektur-Erosion* entsteht durch **Verletzungen** der geplanten Architektur — man bricht bewusst oder unbewusst die vorgesehenen Regeln. *Architektur-Drift* entsteht durch **Unwissen** über die Architektur — die Struktur löst sich auf, weil niemand sie mehr kennt. Beide Phänomene kehren weiter unten am Fallbeispiel wieder: Schicht-Verletzungen sind ein Lehrbuchbeispiel für Erosion, Import-Zyklen ein Symptom von Drift.

David Parnas verband 1994 in *Software Aging* den Verfall direkt mit dem Menschen: eine Hauptursache der Alterung sei *Ignorant Surgery* — Eingriffe durch Entwickler, die das System nicht mehr verstehen, bis es schließlich niemand mehr versteht. Damit ist der Wissensverlust beim Ausscheiden erfahrener Köpfe kein Randthema, sondern eine zentrale technische Gefahr. Eine versionierte, automatisch erzeugte Landkarte konserviert Architekturwissen unabhängig von einzelnen Personen. Wohin das Ganze führt, wenn man nichts tut, beschrieben **Brian Foote und Joseph Yoder** 1997 mit dem anschaulichen Schreckbild des *Big Ball of Mud* — des planlos gewachsenen Systems, in dem Information wahllos zwischen entfernten Teilen geteilt wird und fast alles mit allem verbunden ist.

Damit das für das Management greifbar wird, hilft die Sprache der **technischen Schuld** (*technical debt*), die **Ward Cunningham** 1992 prägte: nicht ganz richtiger Code ist wie aufgenommene Schuld; sie beschleunigt die Entwicklung kurzfristig, doch man zahlt *Zinsen* in Form steigenden Änderungsaufwands, solange man sie nicht tilgt. (Cunningham betonte später, er habe damit *nicht* absichtlich schlampigen Code gemeint, sondern das natürliche Reifen des Verständnisses — die Metapher beschreibt eine Last, keinen Vorwurf.) **John Ousterhout** schliesslich benennt 2018 in *A Philosophy of Software Design* die konkrete Wurzel: Komplexität entsteht vor allem aus **Abhängigkeiten** und Unklarheit, und sie zeigt sich als *Änderungsverstärkung* (eine kleine Änderung erzwingt Anpassungen an vielen Stellen) und als hohe *kognitive Last* bei der Einarbeitung.



- Die obere Reihe sind die klassischen, voneinander unabhängig beschriebenen Grundprobleme.
- Sie alle konkretisieren sich an genau einer Stelle: den **Abhaengigkeiten** zwischen den Bausteinen.
- Das ist der Grund, warum die Abhaengigkeitsanalyse kein Nischenthema ist — sie setzt dort an, wo die grossen Probleme praktisch greifbar werden. Genau diese Groesse macht die Landkarte messbar.

Aus der Praxis (eine reale Referenz-Anwendung): Der praktische Pruefstand dieser Methodik ist eine reale Desktop-Anwendung mit rund 750 internen Modulen, aufgeteilt in einen Kern und mehrere Funktions- und Anwendungsschichten. Diese Anwendung liefert die konkreten, anonymisierten Zahlen, die in den folgenden Kaesten auftauchen.

2. Grundbegriffe (Glossar zum Einstieg)

Einstieg: Die folgenden Begriffe tauchen im Rest der Arbeit auf. Wer sie kennt, kann dieses Kapitel ueberspringen und spaeter zum Nachschlagen zurueckkehren.

- **Modul:** Ein einzelner Baustein des Programms — in vielen Sprachen eine Quelldatei oder ein Paket (ein Verzeichnis von Modulen).
- **Abhaengigkeit / Import:** Modul A *haengt ab von* Modul B, wenn A den Code von B benutzt (etwa per `import`). Im Bild ist das ein Pfeil von A nach B.
- **Kopplung (Coupling):** Wie stark zwei Module aneinander haengen. Ziel ist *lose* Kopplung — wenige, klar umrissene Verbindungen statt vieler verflochtener.
- **Kohaesion (Cohesion):** Wie gut die Bestandteile *innerhalb* eines Moduls zusammengehoeeren. Hohe Kohaesion (ein Modul, eine Aufgabe) fuehrt meist von selbst zu loser Kopplung.
- **Zyklus / zirkulaere Abhaengigkeit:** A braucht B, und B braucht (direkt oder ueber Umwege) wieder A. Der Abhaengigkeitsgraph enthaelt dann einen Kreis.
- **Gerichteter azyklischer Graph (DAG):** Ein Abhaengigkeitsgraph *ohne* Kreise — der gesunde Zielzustand.
- **Stark zusammenhaengende Komponente (SCC):** Eine maximale Gruppe von Modulen, in der jedes jedes andere erreichen kann. Jede SCC mit mehr als einem Modul *ist* genau ein Import-Zyklus.
- **Tarjan-Algorithmus:** Verfahren von Robert Tarjan (1972), das alle SCC eines Graphen mit einer einzigen Tiefensuche in Linearzeit findet — die mathematisch exakte Zyklen-Erkennung.
- **Schicht (Layer) / Schicht-Verletzung:** In einer Schichtenarchitektur duerfen Abhaengigkeiten nur in eine Richtung zeigen (aussen nach innen). Greift eine innere Schicht nach aussen, ist das eine *Schicht-Verletzung*.
- **Dependency Rule:** Regel aus Robert C. Martins *Clean Architecture*: Quellcode-Abhaengigkeiten duerfen nur nach **innen** zeigen — der Kern darf die aeusseren Schichten nicht kennen.

- **ADP / SDP / SAP:** Martins Paket-Prinzipien — *Acyclic Dependencies* (keine Zyklen), *Stable Dependencies* (nur von Stabilerem abhaengen), *Stable Abstractions* (Stabiles soll abstrakt sein).
- **DIP (Dependency Inversion Principle):** Das Reparatur-Werkzeug — man legt eine Abstraktion (ein Interface) in den Kern und die konkrete Umsetzung nach aussen, womit sich die Pfeilrichtung umdreht.
- **Instabilitaet I:** Kennzahl $I = C_e / (C_a + C_e)$ zwischen 0 und 1. C_e = Zahl der Module, von denen ein Modul abhaengt; C_a = Zahl der Module, die von ihm abhaengen. $I=0$ ist maximal stabil, $I=1$ maximal instabil.
- **Hexagonal / Ports und Adapters:** Architekturmuster (Alistair Cockburn, 2005), bei dem der Anwendungskern nur ueber definierte *Ports* spricht und Technik (Datenbank, Krypto, Oberflaeche) aussen als *Adapter* andockt — damit der Kern isoliert testbar bleibt.
- **Statische Analyse:** Untersuchung des Codes, ohne ihn auszufuehren. Die Landkarte liest die Import-Anweisungen, statt das Programm laufen zu lassen.
- **Graphviz (dot) / DOT:** Ein verbreitetes Zeichen-Werkzeug; *DOT* ist die textuelle Beschreibungssprache, aus der das Kommando `dot` den Graphen rendert.
- **Namespace-Paket:** Ein Verzeichnis ohne ausdrueckliche Paket-Markierung, das die Sprache dennoch als Paket behandelt. Aeltere Analyse-Werkzeuge uebersehen solche Pakete leicht.
- **Floating / isoliertes Modul:** Ein Modul ohne sichtbare ein- oder ausgehende Importe. Es kann toter Code sein — *oder* nur dynamisch (zur Laufzeit) geladen und daher statisch unsichtbar.
- **Architektur-Linter:** Ein Werkzeug, das Schicht- und Zyklus-Regeln *erzwingt* (in der Bauautomatik prueft), waehrend die Landkarte sie nur *zeigt*.
- **ADR (Architecture Decision Record):** Kurze Textdatei, die *eine* Architekturentscheidung samt Begruendung festhaelt (Michael Nygard, 2011).
- **SBOM / SCA:** Inventar (*Software Bill of Materials*) bzw. Analyse der *Drittanbieter*-Bibliotheken — die externe Lieferkette, klar zu trennen von der hier behandelten internen Modulstruktur.

3. Was diese Arbeit behandelt — und was nicht

Einstieg: Das Wort *Abhaengigkeit* hat in der Softwarewelt zwei voellig verschiedene Bedeutungen. Ein kurzer Absatz beugt der haeufigsten Verwechslung vor.

Diese Arbeit behandelt **interne** Abhaengigkeiten: *welches eigene Modul importiert welches andere eigene Modul*. Das ist eine Frage der Architektur und Wartbarkeit. Davon strikt zu trennen sind **externe** Abhaengigkeiten: die eingebundenen Fremdbibliotheken, die in einer **SBOM** inventarisiert und per **SCA** (Software Composition Analysis) auf bekannte Schwachstellen und Lizenzkonflikte geprueft werden — eine Frage der Lieferketten-*Sicherheit* (Standardformate hierfuer sind SPDX und OWASP CycloneDX). Beides heisst *Dependency*, adressiert aber gegensaeztliche Probleme. Die externe Lieferkette ist bewusst **nicht** Gegenstand dieser Arbeit.

Hinweis: Merksatz zur Abgrenzung — *interne* Abhaengigkeiten betreffen die **Wartbarkeit** des eigenen Codes; *externe* Abhaengigkeiten betreffen die **Sicherheit** fremden Codes.

4. Wie eine Abhaengigkeits-Landkarte aussieht

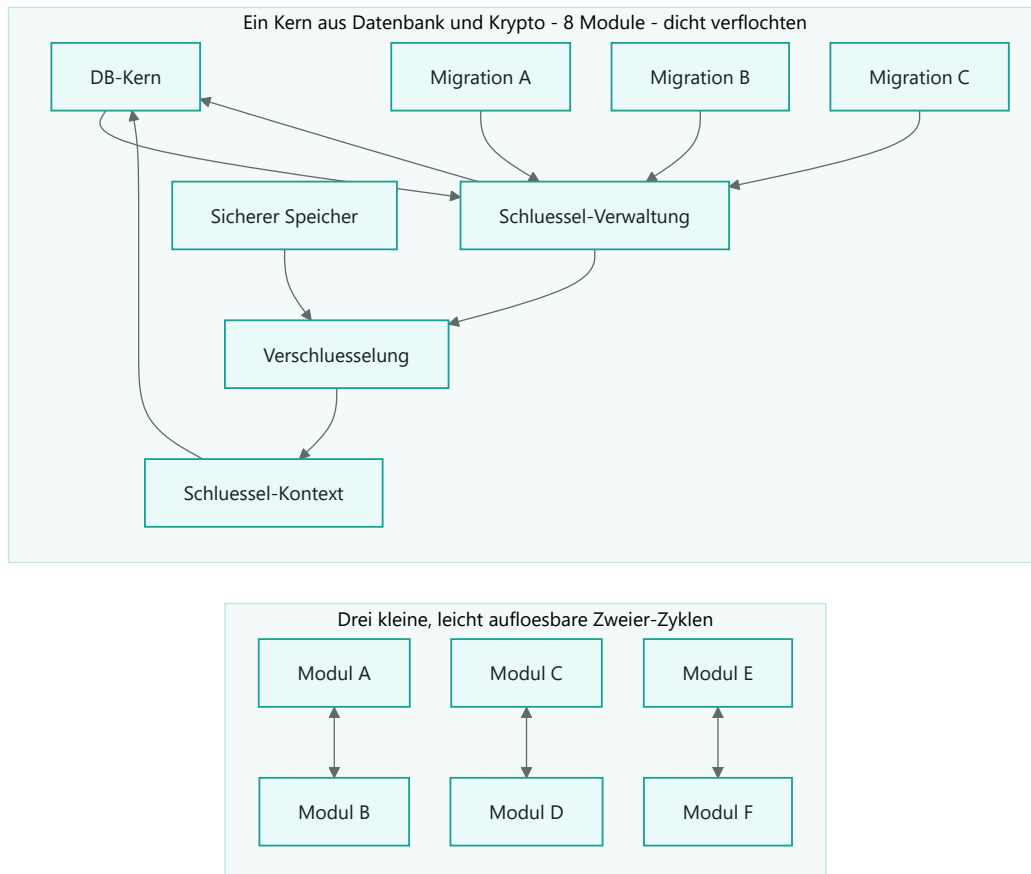
Einstieg: Dieses Kapitel beantwortet die naheliegendste Frage zuerst — wie sieht so eine Karte konkret aus? — und fuehrt das anonymisierte Fallbeispiel ein, das als roter Faden durch die folgenden Kapitel laeuft.

Ein typisches Werkzeug erzeugt aus einer Codebasis drei Artefakte: eine **Overview-Grafik** (alle Module, nach Top-Paket farbig geclustert), eine separate **Zyklen-Grafik** (nur die an Kreisen beteiligten Module, hervorgehoben) und eine **textuelle Zusammenfassung** mit den Kennzahlen, den Zyklen samt Dateipfaden und den Schicht-Verletzungen.

Aus der Praxis (anonymisierte Kennzahlen): Der Lauf auf der realen Referenz-Anwendung ergab folgendes Bild.

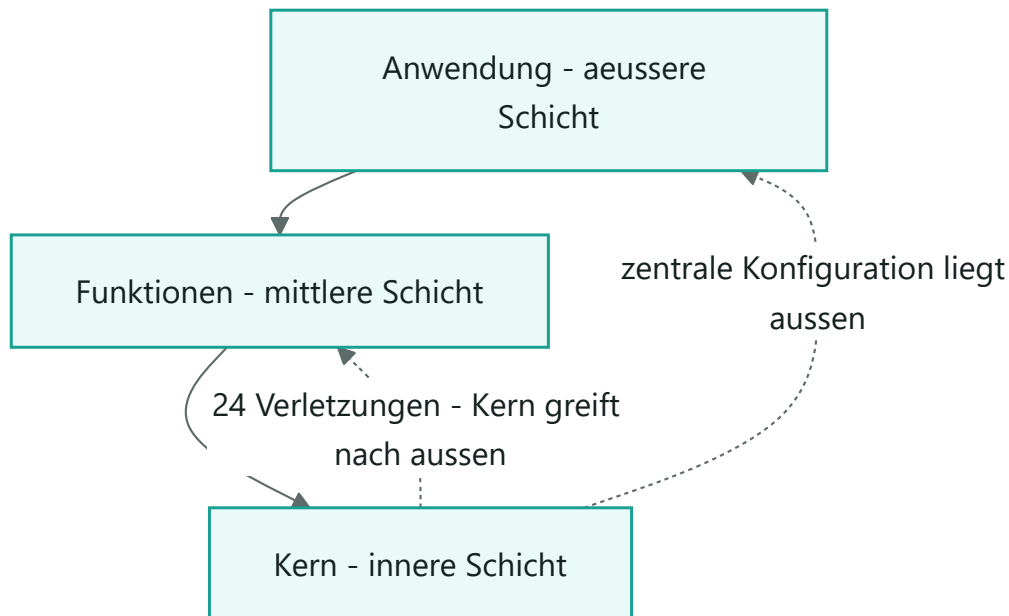
Kennzahl	Wert
Interne Module	rund 750 (ein kleiner Anwendungs-, ein grosser Kern- und ein sehr grosser Funktions-Bereich)
Interne Import-Kanten	rund 2000
Import-Zyklen	4
Schicht-Verletzungen (innen greift nach aussen)	24
Floating / isolierte Module	1 (zzgl. leerer Paket-Markierungen)

Die **Overview-Grafik** ist bei mehreren hundert Knoten bewusst kein Schoenbild: Sie zeigt einen dichten, nach Schichten eingefaeerbten Block und kommuniziert vor allem *Massstab und Cluster-Dichte* — woran man sofort sieht, welche Schicht den Loewenanteil traegt und ob der Kern ungewoehnlich gross geraten ist. Die eigentliche Diagnose liefert die **Zyklen-Grafik**. Sie enthielt im Fallbeispiel drei harmlose Zweier-Paare und einen echten Problemkern:



- Die drei **Zweier-Zyklen** (oben) sind enge Gegenseitig-Importe; ein gemeinsamer Typ oder ein kleiner Service-Schnitt löst sie auf. Geringer Aufwand.
- Der **Acht-Modul-Kern** (unten) aus Datenbank- und Kryptographie-Modulen ist ein echtes Geflecht: diese acht Module lassen sich faktisch nur noch *gemeinsam* testen und ausliefern. Das Diagramm ist schematisch — entscheidend ist die Dichte, nicht die genaue Kantenführung.

Die 24 **Schicht-Verletzungen** betreffen Kanten, bei denen die innere Kernschicht nach aussen in die Funktions- oder Anwendungsschicht greift — typisch etwa, wenn der Kern direkt auf eine zentrale Konfiguration zugreift, die eigentlich aussen liegt. Im Soll-Bild duerften die Pfeile nur nach innen zeigen:



- Durchgezogene Pfeile zeigen die **erlaubte** Richtung (aus innen haengt von innen ab).
- Die gepunkteten Pfeile sind die **Verletzungen**: der Kern macht sich von aeusseren, haeufiger wechselnden Schichten abhaengig — genau die Erosion aus dem Vorwort.

Beispiel: Das eine `floating` Modul hat keinerlei sichtbare Import-Kanten. Das heisst *nicht* automatisch, dass es toter Code ist — es koennte nur dynamisch geladen werden (siehe Kapitel 7). Die Landkarte stellt die Frage; beantworten muss sie ein Mensch.

5. Nutzen — warum sich Abhaengigkeitsanalyse lohnt

Einstieg: Eine Karte ist nur so viel wert wie die Entscheidungen, die sie ermoeeglicht. Dieses Kapitel begruendet den Nutzen in drei Stufen: theoretisch (warum die Befunde schaden), empirisch (dass es real teuer ist) und dokumentarisch (was die Karte als lebendiges Dokument leistet).

5.1 Theorie: warum Zyklen und Schicht-Verletzungen schaden

Die Befunde sind kein Geschmacksurteil, sondern Verstoesse gegen etablierte Prinzipien. Robert C. Martins **Acyclic Dependencies Principle** verlangt, dass der Abhaengigkeitsgraph ein DAG ohne Kreise ist — denn Module in einem Zyklus lassen sich nicht mehr isoliert testen, kompilieren, verstehen oder wiederverwenden; sie verhalten sich wie *ein* untrennbarer Block. Das **Stable Dependencies Principle** erklart, warum die Schicht-Verletzungen besonders heikel sind: Abhaengigkeiten sollen in Richtung wachsender Stabilitaet zeigen, doch ein Kern, der nach aussen auf haeufiger geaenderten Code greift, wird ungewollt selbst instabil. Schon **John Lakos** zeigte 1996 (am Beispiel grosser C++-Systeme), dass azyklische, *levelisierbare* Strukturen nachweislich leichter zu testen, zu warten und wiederzuverwenden sind. Das passende Reparatur-Werkzeug ist das **Dependency Inversion Principle** (siehe Kapitel 6.4).

5.2 Empirie: dass es real teuer ist

Es bleibt nicht bei der Theorie. **Melton und Tempero** fanden 2007 in einer Studie an 78 Java-Anwendungen, dass rund 45 Prozent einen Zyklus mit mindestens 100 Klassen und etwa 10 Prozent sogar einen mit ueber 1000 Klassen enthielten — kleine Zyklen wachsen erfahrungsgemaess zu riesigen, unwartbaren Clustern. **Oyetoyan und Kollegen** wiesen 2015 eine positive Korrelation zwischen zyklischer Abhaengigkeit und Aenderungshaeufigkeit nach: Code in der Naehe von Zyklen muss messbar oefter angefasst werden und verursacht einen ueberproportionalen Anteil aller Aenderungen.

Hinweis: Ehrlichkeit gebietet zwei Einschränkungen. Erstens stammen diese Befunde aus **Java**-Codebasen; die Uebertragung auf andere Sprachen ist ein plausibler Analogieschluss, aber nicht fuer jede Sprache eigens bewiesen. Zweitens zeigt eine differenzierende Gegenstimme — **Al-Mutawa und Kollegen** (2014) —, dass *nicht jeder* Zyklus gleich schaedlich ist: viele sind klein und lokal. Die praktische Konsequenz ist keine Null-Toleranz, sondern **Priorisierung** nach Groesse und Schicht-Relevanz. Genau deshalb verdient ein grosser, schichtuebergreifender Kern-Cluster Aufmerksamkeit, ein enges Zweier-Paar dagegen nur einen kleinen Handgriff.

5.3 Dokumentations-Nutzen: ein lebendiges, driftfreies Artefakt

Der vielleicht groesste Nutzen ist dokumentarisch. Ein handgezeichnetes Architekturbild beginnt zu *luegen*, sobald der erste Commit nach seiner Erstellung erfolgt — dieses Auseinanderlaufen heisst *Documentation Drift*. Eine aus dem Code erzeugte Landkarte kann diesen Drift konstruktionsbedingt nicht entwickeln, weil sie ihre Wahrheit aus dem Quelltext selbst zieht (das Prinzip *Diagrams as Code*: gleiche Eingabe, gleiche Ausgabe, kein manuelles Verschieben von Kaesten). Sie ist damit ein *lebendiges Dokumentationsartefakt* im Sinne von Cyrille Martraires *Living Documentation* (2019). In etablierte Rahmen eingeordnet:

- Im **C4-Modell** (Simon Brown) entspricht die Overview-Grafik der *Component*-Ebene.
- Formal ist die Karte eine *View* im Sinne von **ISO/IEC/IEEE 42010** (Architekturbeschreibung) fuer das Anliegen "Modulkopplung".
- Sie fuellt zwei Schubladen des deutschsprachigen Standards **arc42**: Abschnitt 5 (Bausteinsicht) mit der Karte selbst, Abschnitt 11 (Risiken und technische Schulden) mit den Zyklen und Verletzungen.
- Und sie macht den Begriff *Qualitaet* normativ greifbar: **ISO/IEC 25010** definiert *Wartbarkeit* (Maintainability) ueber die Sub-Merkmale Modularitaet, Wiederverwendbarkeit, Analysierbarkeit und Aenderbarkeit — exakt das, was eine Abhaengigkeits-Landkarte misst.

Aspekt	Aufwand	Ertrag
Einmaliger Lauf	Minuten (Open-Source-Werkzeuge)	Vollstaendiges, objektives Ist-Bild der Kopplung
Versioniertes Artefakt	gering (ein Build-Schritt)	Architektur-Aenderungen werden im Pull-Request diffbar
Bauautomatik-Schranke mit Baseline	einmalig einrichten	Neue Zyklen/Verletzungen werden automatisch verhindert
Onboarding und Audit	keiner zusaetzlich	Schnelleres Einarbeiten; belastbarer Nachweis fuer Reviewer/Auditoren

Gerade fuer **sicherheitskritische** Software wiegt der Audit-Nutzen schwer: Eine versionierte Karte dokumentiert objektiv, welche Module welche erreichen koennen, und ungenutzter (toter) Code vergroessert unnoetig die Angriffsflaeche.

6. Einrichtung — vom ersten Lauf zum dauerhaften Schutz

Einstieg: Dieses Kapitel ist das praktische "Wie". Es fuehrt von den Voraussetzungen ueber den konkreten Lauf bis zur dauerhaften Verankerung in der Bauautomatik (CI) und zur Reparatur der gefundenen Probleme.

6.1 Voraussetzungen

Es gibt zwei harte Voraussetzungen. Erstens muss der Code **analysierbar** sein — das Werkzeug betrachtet nur Module, die es ueber die regulaere Import-Aufloesung der Sprache findet. Zweitens muss ein **Graph-Zeichner** (etwa Graphviz mit dem Kommando `dot`) installiert und im Suchpfad sein, um die Grafik zu rendern.

Achtung: Ein haeufiger Erstfehler lautet sinngemaess `dot not found`. Ursache: Die blosse Sprach-Anbindung eines Zeichen-Werkzeugs installiert oft *nicht* die eigentliche Render-Engine; letztere kommt ueber den Paketmanager des Betriebssystems.

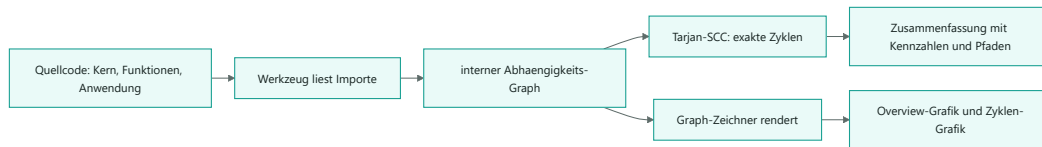
Ein wiederkehrender Stolperstein sind **Namespace-Pakete** — Verzeichnisse ohne ausdrueckliche Paket-Markierung. Aeltere Werkzeuge uebersehen die Module darunter. Robuste Werkzeuge umgehen das, indem sie solche Verzeichnisse fuer die Dauer der Analyse *transient* paketieren, damit absolute Importe korrekt auflösen.

6.2 Der Lauf

Ein gutes Werkzeug ist mit einer einzigen Befehlszeile bedienbar: Man uebergibt das Repository und die zu kartierenden Top-Pakete

von innen nach aussen

(der am meisten genutzte Kern zuerst), damit das Werkzeug die Schicht-Verletzungen daraus ableiten kann. Intern liest es die Importe, baut daraus den Graphen, ermittelt mit Tarjans Algorithmus die Zyklen und laesst den Graph-Zeichner die Grafiken erzeugen:



- Die Zyklen-Erkennung ist kein Schaetzverfahren, sondern der bewiesen korrekte Tarjan-Algorithmus in Linearzeit.
- Als Ausgabeformat eignet sich **SVG**: Es skaliert verlustfrei und laesst sich gut in Dokumente und Wikis einbetten.

6.3 Lesbarkeit bei grossen Codebasen

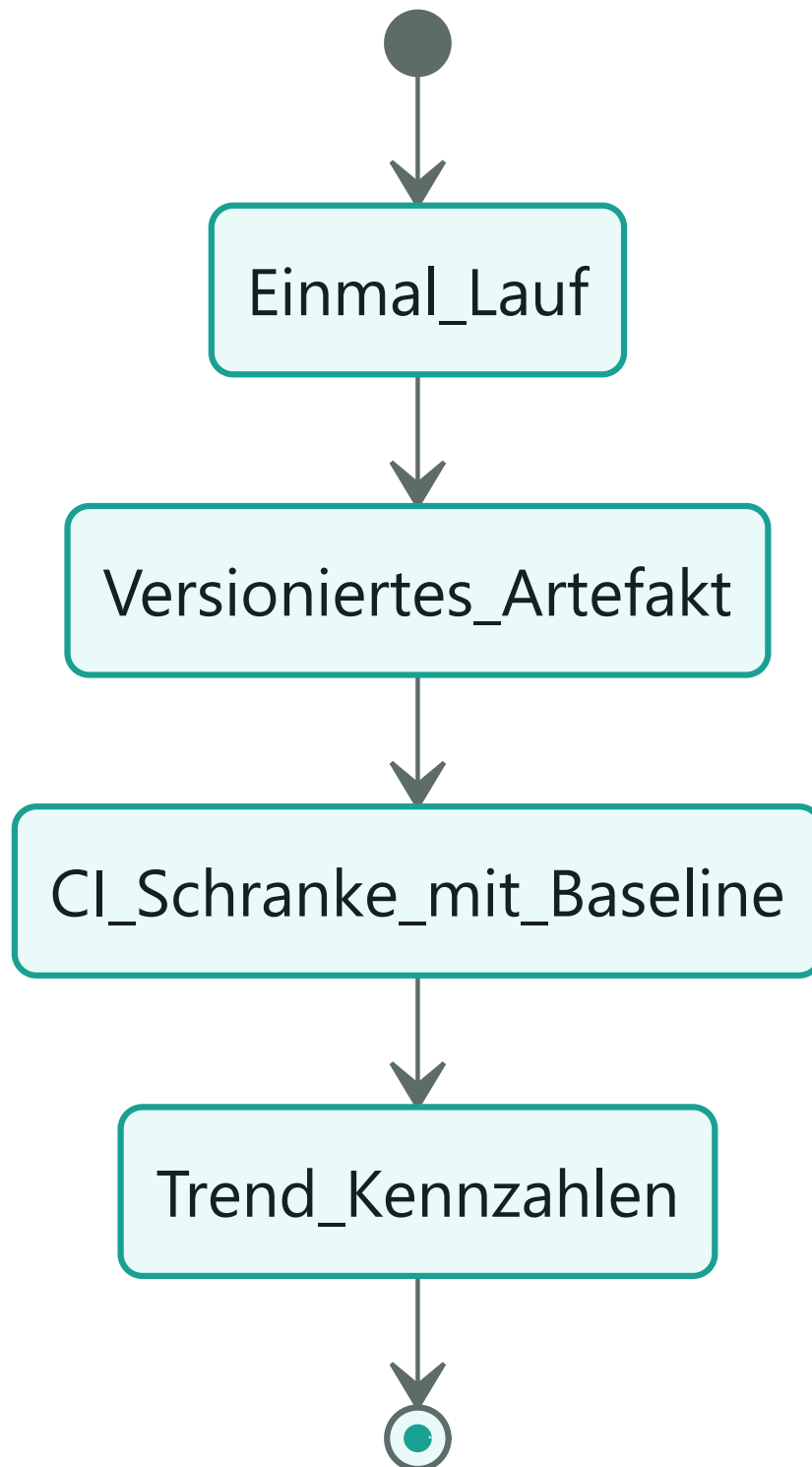
Bei vielen hundert Modulen wird ein flacher Graph zum unlesbaren "Hairball". Gute Werkzeuge begegnen dem, indem sie die Overview-Grafik nach Top-Paket *clustern* und ihre Tiefe begrenzen, waehrend die Zyklen stets voll aufgeloest bleiben. Fuer noch groessere Systeme ist die **Dependency Structure Matrix** (eine quadratische Tabellendarstellung) die skalierende Alternative — sie macht Zyklen als Eintraege ober- und unterhalb der Diagonale sichtbar und erlaubt hierarchisches Drill-down.

6.4 Reparatur der Befunde

Das Zielbild fuer einen verflochtenen Datenbank-/Krypto-Kern liefert die **Hexagonale Architektur**: Datenbank und Kryptographie gehoeren hinter klar definierte Ports und Adapter, nicht in den Kern. Konkret bricht das **Dependency Inversion Principle** die Verletzungen auf — man legt das benoetigte *Interface* in die innere Schicht und verlagert die konkrete Implementierung nach aussen, womit sich die Pfeilrichtung umdreht, ohne dass sich das Verhalten aendert. Bei gewachsenem Code geschieht das **schrittweise** und mit Sicherheitsnetz — im Sinne von Michael Feathers' *Working Effectively with Legacy Code* (2004): erst Charakterisierungstests, dann den Zyklus an seiner schwachsten Kante aufschneiden.

6.5 Dauerbetrieb: von der Momentaufnahme zur Ratsche

Der groesste Nutzen entsteht nicht durch den Einmal-Lauf, sondern durch dauerhafte Durchsetzung. Hier ist die Rollenteilung wichtig: Die Landkarte ist das Bild **fuer Menschen**; ein dedizierter **Architektur-Linter** ist die **Schranke fuer die Bauautomatik**, die Schicht- und Zyklus-Regeln deklarativ prueft und den Build bricht, sobald jemand eine neue Verletzung einfuehrt. Eingefuehrt wird das nicht als Big Bang, sondern als Reifegradleiter mit einer **Baseline** (der Ist-Stand wird als bekannte Ausnahme eingefroren; nur *neue* Verstoesse failen, und die erlaubte Anzahl darf nur sinken — eine Ratsche):



- **Stufe 0 — Einmal-Lauf:** Der Ist-Zustand wird sichtbar.
- **Stufe 1 — versioniertes Artefakt:** Karte und Zusammenfassung liegen im Repository; jede Architektur-Änderung wird im Pull-Request diffbar.
- **Stufe 2 — CI-Schranke mit Baseline:** Nur *neue* Zyklen/Verletzungen brechen den Build; Altlasten werden geduldet, aber eingefroren.
- **Stufe 3 — Trend-Kennzahlen:** Zyklen, Verletzungen und floating-Module werden über die Zeit verfolgt — die Deckelung darf nur sinken, die Architektur kann sich messbar nur noch verbessern.

Jede bewusste Entscheidung dabei — einen Zyklus tolerieren, kapseln oder auflösen — gehoert als **ADR** (Architecture Decision Record, Michael Nygard 2011) festgehalten, damit das *Warum* auch Jahre spaeter nachvollziehbar bleibt. Karte und ADR ergaenzen sich: die Karte zeigt das *Was*, der ADR konserviert das *Warum*.

7. Grenzen und Offenes (ehrlich)

Einstieg: Eine vorwissenschaftliche Arbeit benennt ihre Grenzen. Die statische Abhaengigkeitsanalyse ist maechtig, aber sie ist ein *unvollstaendiger Spiegel* — die folgenden Punkte muss man kennen, um aus der Karte keine falschen Schluesse zu ziehen.

- **Dynamische Verbindungen bleiben unsichtbar (False Negatives).** Die Analyse sieht nur, was im Code als Import steht. Vier Klassen von Verbindungen entgehen ihr grundsaeztlich: dynamische, zur Laufzeit aus Zeichenketten aufgeloeoste Importe; Plugin- und Registry-Mechanismen ueber Entry-Points; Dependency-Injection-Container mit Laufzeit-Verdrahtung sowie string-/reflektionsbasierte Router. Ein als `floating` gemeldetes Modul kann daher sehr wohl verbunden sein — es wird nur zur Laufzeit geladen.
 - **Schein-Abhaengigkeiten sind moeglich (False Positives).** Importe, die nur fuer die statische Typpruefung existieren und zur Laufzeit nie ausgefuehrt werden, koennen je nach Werkzeug als echte Kante gezaehlt werden und so *Schein-Zyklen* erzeugen. Gerade sauber getypte Projekte stolpern hier.
 - **Paradoxer Effekt sauberer Architektur.** Je mehr ein System Dependency Injection und Plugins nutzt, desto *leerer* kann sein statischer Graph wirken — obwohl es real dicht verdrahtet ist. Die Karte unterschaezt also gerade gut entkoppelte Systeme.
 - **Empirie ist sprachgebunden.** Die zitierten Studien untersuchten Java-Codebasen. Die Uebertragung auf andere Sprachen ist plausibel, aber ein Analogieschluss, kein Beweis.
 - **Nicht jeder Zyklus ist gleich kritisch.** Eine Null-Toleranz-Haltung waere falsch (vgl. Kapitel 5.2). Die Befunde muessen *triagiert* werden: echte Laufzeit-Verletzung, tolerierbar, oder Werkzeug-Artefakt — nach Groesse und Schicht-Relevanz.
 - **Die Karte ersetzt kein Urteil.** Sie ist ein wertvoller, aber unvollstaendiger Spiegel und ersetzt weder Laufzeit-Tracing noch Code-Review — sie *fokussiert* beides.
-

8. Fazit

Einstieg: Zum Abschluss der rote Faden, verdichtet auf seine Kernaussage.

Die grossen Grundprobleme der Softwareentwicklung — Komplexitaet und Unsichtbarkeit (Brooks), unausweichlicher Verfall (Lehman), Erosion und Drift (Perry/Wolf), Alterung durch Wissensverlust (Parnas), das Abgleiten zum Big Ball of Mud (Foote/Yoder), die wachsende technische Schuld (Cunningham) — konkretisieren sich an einer einzigen, messbaren Stelle: den Abhaengigkeiten

zwischen den Modulen. Die Abhaengigkeits-Landkarte greift genau dort an. Sie macht die verborgene Struktur sichtbar, sie belegt mit etablierten Prinzipien und empirischer Forschung, dass Zyklen und Schicht-Verletzungen real teuer sind, und sie tut dies als *lebendiges*

, aus dem Code erzeugtes Dokument, das — anders als jedes handgemalte Bild — gar nicht erst veralten kann. Im anonymisierten Fallbeispiel hat sie aus rund 750 Modulen vier Zyklen und 24 Verletzungen herausgehoben und damit aus einem diffusen Unbehagen eine konkrete, priorisierbare Aufgabenliste gemacht. Der gemeinsame Nenner ueber alle Kapitel hinweg ist ein einziges Prinzip:

Was man nicht sieht, kann man nicht steuern — und eine aus dem Code erzeugte Karte veraltet nie.

9. Quellen und weiterfuehrende Grundlagen

Einstieg: Die Arbeit stuetzt sich auf anerkannte Forschung und Standards. Die Jahreszahlen folgen, wo noetig, den Erstveroeffentlichungen.

Grundprobleme:

- Frederick P. Brooks Jr. (1986/1987): *No Silver Bullet — Essence and Accidents of Software Engineering*. IEEE Computer 20(4):10-19.
- Meir M. Lehman (1980): *Programs, Life Cycles, and Laws of Software Evolution*. Proc. IEEE 68(9):1060-1076 (erste Gesetze 1974).
- Dewayne E. Perry, Alexander L. Wolf (1992): *Foundations for the Study of Software Architecture*. ACM SIGSOFT SEN 17(4):40-52 (Erosion vs. Drift).
- David L. Parnas (1994): *Software Aging*. ICSE-16, S. 279-287.
- Brian Foote, Joseph Yoder (1997): *Big Ball of Mud*. PLoP '97.
- Ward Cunningham (1992): *The WyCash Portfolio Management System*. OOPSLA '92 (Ursprung der Technical-Debt-Metapher); Martin Fowler (2009): *Technical Debt Quadrant*.
- John Ousterhout (2018): *A Philosophy of Software Design*.

Kopplung, Zyklen, Prinzipien:

- W. P. Stevens, G. J. Myers, L. L. Constantine (1974): *Structured Design*. IBM Systems Journal 13(2) (Kopplung und Kohaesion).
- Robert E. Tarjan (1972): *Depth-First Search and Linear Graph Algorithms*. SIAM J. Comput. 1(2):146-160 (SCC in $O(V+E)$).
- Robert C. Martin (1994): *OO Design Quality Metrics (Ca/Ce/I/A/D)*; (2000): *Design Principles and Design Patterns (ADP/SDP/SAP/DIP)*; (2017): *Clean Architecture (Dependency Rule)*.
- John Lakos (1996): *Large-Scale C++ Software Design (Levelisierung)*.
- Alistair Cockburn (2005): *Hexagonal (Ports and Adapters) Architecture*.
- Empirie (Java): Melton & Tempero (2007), *Empirical Software Engineering* 12:389-415; Oyetoyan et al. (2015), SANER; Al-Mutawa et al. (2014), ASWEC (differenzierende Gegenstimme).
- SEI CERT Oracle Coding Standard for Java, Regel DCL60-J (Paketzyklen vermeiden).

Nutzen, Dokumentation, Standards:

- Neal Ford, Rebecca Parsons, Patrick Kua (2017): *Building Evolutionary Architectures* (Architektur-Fitness-Funktionen).
- Simon Brown (2011 ff.): *C4-Modell* (c4model.com).
- Michael Nygard (2011): *Documenting Architecture Decisions* (ADR).
- Gernot Starke, Peter Hruschka (2005 ff.): *arc42*.
- ISO/IEC/IEEE 42010:2022 — Architecture description (Views); ISO/IEC 25010 — Quality models (Maintainability).
- Cyrille Martraire (2019): *Living Documentation*. — Daniele Procida: *Diátaxis* (Explanation-Quadrant).

Werkzeuge und Einrichtung:

- Graph-Extraktion und -Rendering: Graphviz / DOT (Gansner, Koutsofios, North); fuer Python verbreitet etwa pydeps zur Import-Extraktion.
- Architektur-Linter zur Durchsetzung in der Bauautomatik (z. B. import-linter oder tach im Python-Umfeld; ArchUnit im Java-Umfeld; dependency-cruiser im JS/TS-Umfeld).
- Michael C. Feathers (2004): *Working Effectively with Legacy Code* (inkrementelles Aufbrechen).
- Abgrenzung externe Lieferkette: SPDX; OWASP CycloneDX (SBOM/SCA).